

ФГБОУ ВО
Уфимский университет науки и технологий
Кафедра Технической Кибернетики

Отчет
по практической работе №2
по дисциплине «Сетевые технологии»

Выполнил: ст. гр.
ИВТ-ПО-401Б
Данилова А. А.
Принял: к.т.н., к.ф.н.
Насыров Р. В.

Уфа 2026 г.

Этап 1. Создание виртуальной среды (Virtualenv)

Виртуальная среда — это изолированная среда разработки, в которой хранятся зависимости приложения (установленные пакеты). Среда создавалась в папке todos.

В терминале выполнена команда создания папки и перехода в неё: `mkdir todos && cd todos`. Затем создана виртуальная среда командой: `python3 -m venv venv`. В результате внутри папки появилась папка `venv`, в которой находятся папки `bin` и `lib`: в `lib` хранится копия интерпретатора Python, в `bin` — файлы, обеспечивающие работу среды.

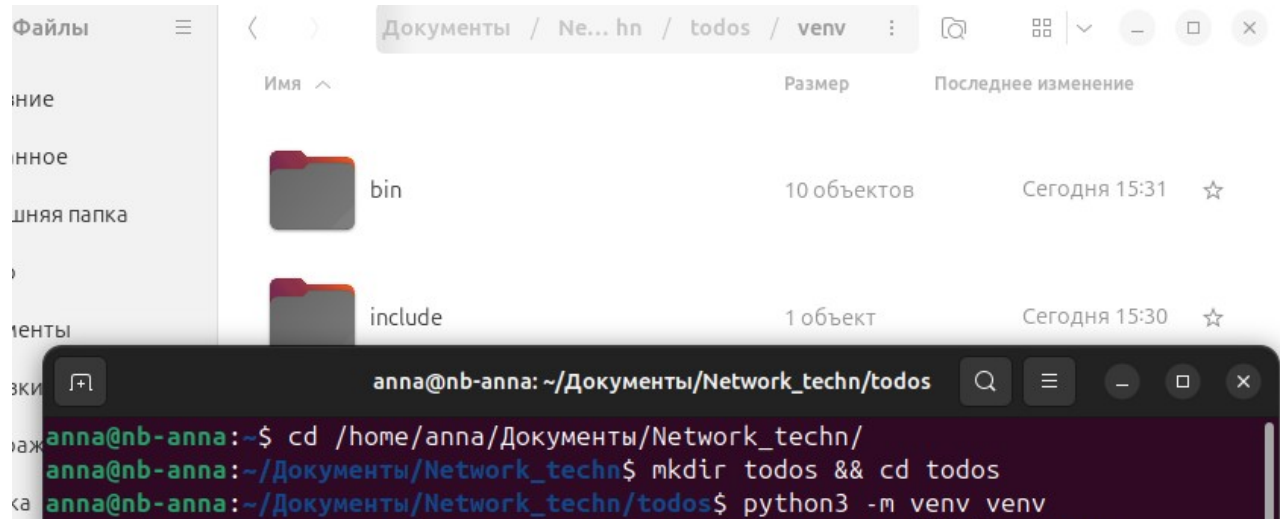


Рисунок 1. Создание папки проекта и виртуальной среды

Для работы внутри виртуальной среды её необходимо активировать командой: `source venv/bin/activate`. После активации перед приглашением терминала появляется префикс `(venv)`.

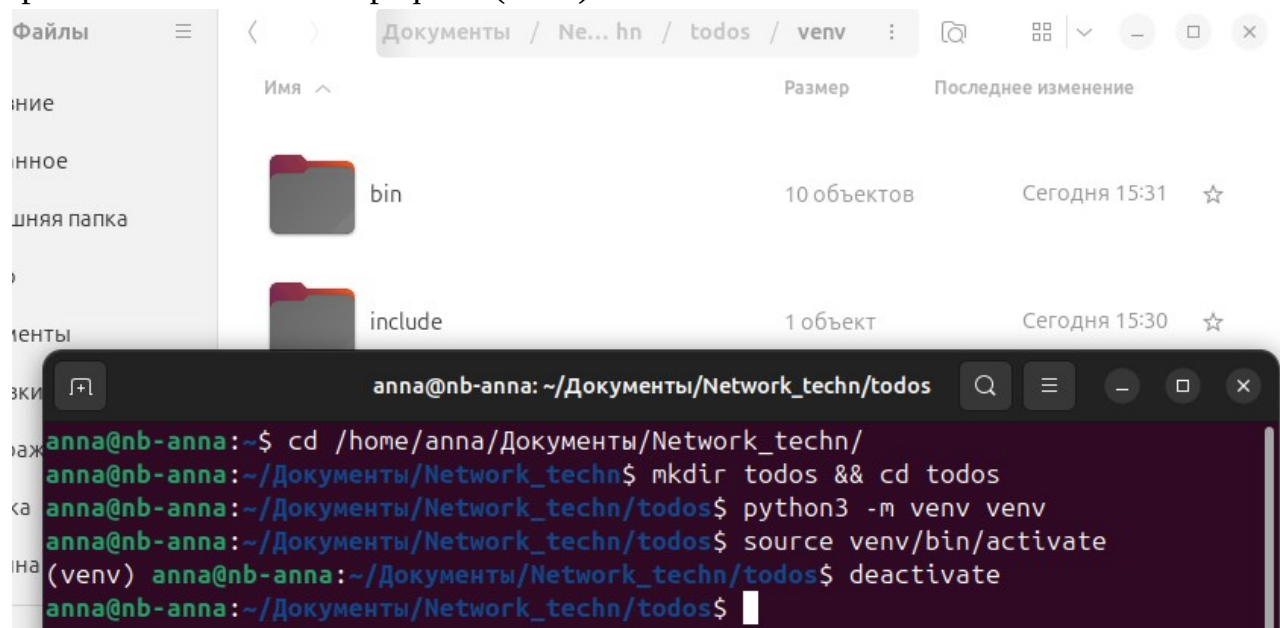


Рисунок 2. Активация виртуальной среды

Этап 2. Управление пакетами с помощью pip

pip — это менеджер пакетов Python. Он автоматизирует установку и удаление пакетов. Команда `pip list` выводит список установленных пакетов. Команда `pip install fastapi` устанавливает пакет FastAPI — фреймворк, на котором строится приложение.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ python3 -m pip list
Package Version
-----
pip      24.0
```

Рисунок 3. Список установленных пакетов

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ pip install fastapi
Collecting fastapi
  Downloading fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting starlette==0.46.0 (from fastapi)
  Downloading starlette-1.6.0-py3-none-any.whl.metadata (6.4 kB)
Collecting pydantic>=2.9.0 (from fastapi)
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
    110.2/110.2 kB 637.5 kB/s eta 0:00:00
Collecting typing-extensions>=4.8.0 (from fastapi)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from fastapi)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc==0.0.2 (from fastapi)
  Downloading annotated_doc-0.0.5-py3-none-any.whl.metadata (6.5 kB)
Collecting annotated-types==0.6.0 (from pydantic>=2.9.0->fastapi)
  Downloading annotated_types-0.8.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.46.5 (from pydantic>=2.9.0->fastapi)
  Downloading pydantic_core-2.46.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)
Collecting anyio<5,>=3.6.2 (from starlette==0.46.0->fastapi)
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.7 kB)
Collecting idna==2.8 (from anyio<5,>=3.6.2->starlette==0.46.0->fastapi)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
  Downloading fastapi-0.141.1-py3-none-any.whl (131 kB)
    132.0/132.0 kB 956.7 kB/s eta 0:00:00
  Downloading annotated_doc-0.0.5-py3-none-any.whl (5.3 kB)
  Downloading pydantic-2.13.5-py3-none-any.whl (472 kB)
    472.6/472.6 kB 1.4 MB/s eta 0:00:00
  Downloading pydantic_core-2.46.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.1 MB)
    2.1/2.1 MB 1.9 MB/s eta 0:00:00
  Downloading starlette-1.6.0-py3-none-any.whl (75 kB)
    76.0/76.0 kB 2.0 MB/s eta 0:00:00
  Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
  Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
  Downloading annotated_types-0.8.0-py3-none-any.whl (13 kB)
  Downloading anyio-4.15.1-py3-none-any.whl (132 kB)
    132.1/132.1 kB 1.7 MB/s eta 0:00:00
  Downloading idna-3.20-py3-none-any.whl (69 kB)
    69.6/69.6 kB 1.7 MB/s eta 0:00:00
Installing collected packages: typing-extensions, idna, annotated-types, annotated-doc, typing-inspection, pydantic-core, anyio, starlette, pydantic, fastapi
Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 fastapi-0.141.1 idna-3.20 pydantic-2.13.5 pydantic-core-2.46.5 starlette-1.6.0 typing-extensions-4.16.0 typing-inspection-0.4.4
```

Рисунок 4. Установка пакета *fastapi*

Команда `pip freeze > requirements.txt` сохраняет список установленных пакетов в файл `requirements.txt`. Содержимое файла просматривается командой `cat requirements.txt`.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ pip freeze > requirements.txt
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ cat requirements.txt
annotated-doc==0.0.5
annotated-types==0.8.0
anyio==4.15.1
fastapi==0.141.1
idna==3.20
pydantic==2.13.5
pydantic_core==2.46.5
starlette==1.6.0
typing-inspection==0.4.4
typing_extensions==4.16.0
(venv) anna@nb-anna:~/Документы/Network_techn/todos$
```

Рисунок 5. Список пакетов в файле *requirements.txt*

Этап 3. Настройка Docker

Docker — инструмент для контейнеризации приложений. Файл Dockerfile содержит инструкции, по которым собирается образ приложения. Для примера создан файл hello.py, а Dockerfile содержит строки: FROM python:3.8; WORKDIR /usr/src/app; ADD . /usr/src/app; CMD ["python", "hello.py"].

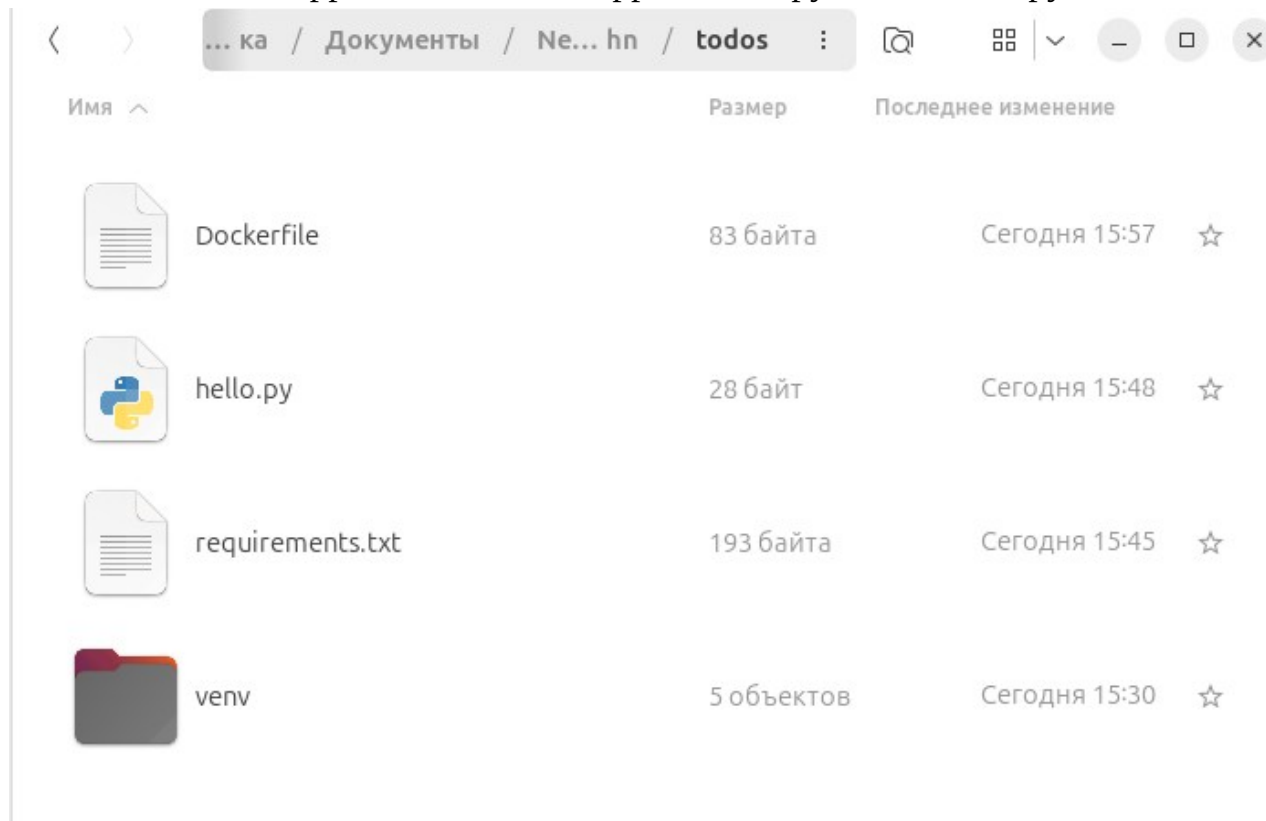


Рисунок 6. Содержимое папки проекта (Dockerfile, hello.py)

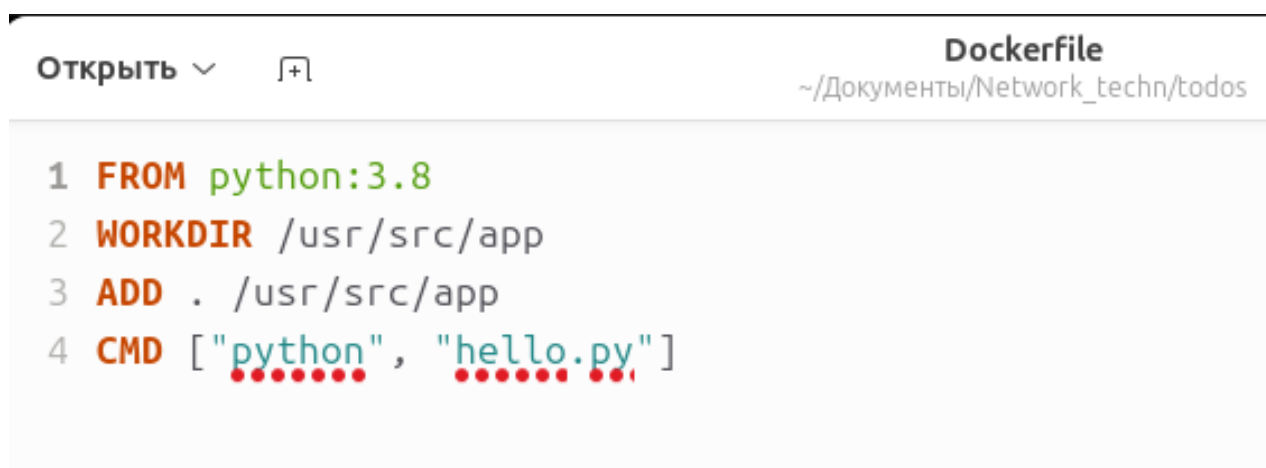


Рисунок 7. Содержимое файла Dockerfile

Образ собирается командой: docker build -t getting_started . Команда загружает образ Python и выполняет инструкции из Dockerfile.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ docker build -t getting_started .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 27.96MB
Step 1/4 : FROM python:3.8
3.8: Pulling from library/python
5a98c896c047: Pulling fs layer
a173f2aee8e9: Pulling fs layer
cdd62bf39133: Pulling fs layer
a47cff7f31e9: Pulling fs layer
cddc73e4e6c7: Pulling fs layer
01272fe8adba: Pulling fs layer
cc48f13b5f0f: Pulling fs layer
22750c0e3d1c: Download complete
81b2fc4bdb7f: Download complete
a47cff7f31e9: Download complete
cddc73e4e6c7: Download complete
5a98c896c047: Download complete
cdd62bf39133: Download complete
a173f2aee8e9: Download complete
cc48f13b5f0f: Download complete
01272fe8adba: Download complete
cdd62bf39133: Pull complete
a47cff7f31e9: Pull complete
a173f2aee8e9: Pull complete
01272fe8adba: Pull complete
cddc73e4e6c7: Pull complete
cc48f13b5f0f: Pull complete
5a98c896c047: Pull complete
Digest: sha256:d411270700143fa2683cc8264d9fa5d3279fd3b6afff62ae81ea2f9d070e390c
Status: Downloaded newer image for python:3.8
--> d41127070014
Step 2/4 : WORKDIR /usr/src/app
--> Running in f32c59c839b1
```

Рисунок 8. Сборка образа Docker

Запуск контейнера выполняется командой: `docker run getting_started`. Приложение выводит текст `Hello from Docker!`. Это значит, что контейнер собран и работает корректно.

```
--> 9ced131488e0
Successfully built 9ced131488e0
Successfully tagged getting_started:latest
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ docker run getting_started
Hello from Docker!
```

Рисунок 9. Запуск контейнера Docker

Этап 4. Создание приложения FastAPI

FastAPI — это веб-фреймворк Python для создания API. В виртуальной среде установлены зависимости: `pip install fastapi uvicorn`. Пакет `uvicorn` запускает сервер приложения.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ pip install fastapi uvicorn
Requirement already satisfied: fastapi in ./venv/lib/python3.12/site-packages (0.141.1)
Collecting uvicorn
  Downloading uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: starlette>=0.46.0 in ./venv/lib/python3.12/site-packages (from fastapi) (1.6.0)
Requirement already satisfied: pydantic>=2.9.0 in ./venv/lib/python3.12/site-packages (from fastapi) (2.13.5)
Requirement already satisfied: typing-extensions>=4.8.0 in ./venv/lib/python3.12/site-packages (from fastapi) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in ./venv/lib/python3.12/site-packages (from fastapi) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in ./venv/lib/python3.12/site-packages (from fastapi) (0.0.5)
Collecting click>=7.0 (from uvicorn)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting h11>=0.8 (from uvicorn)
  Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: annotated-types>=0.6.0 in ./venv/lib/python3.12/site-packages (from pydantic>=2.9.0->fastapi) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in ./venv/lib/python3.12/site-packages (from pydantic>=2.9.0->fastapi) (2.46.5)
Requirement already satisfied: anyio<5,>=3.6.2 in ./venv/lib/python3.12/site-packages (from starlette>=0.46.0->fastapi) (4.15.1)
Requirement already satisfied: idna>=2.8 in ./venv/lib/python3.12/site-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (3.20)
Downloading uvicorn-0.54.0-py3-none-any.whl (87 kB)
87.4/87.4 kB 481.1 kB/s eta 0:00:00
Downloading click-8.5.0-py3-none-any.whl (125 kB)
125.3/125.3 kB 1.8 MB/s eta 0:00:00
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Installing collected packages: h11, click, uvicorn
Successfully installed click-8.5.0 h11-0.16.0 uvicorn-0.54.0
```

Рисунок 10. Установка зависимостей fastapi и uvicorn

Создан файл api.py со следующим кодом:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def welcome() -> dict:
    return {"message": "Hello World"}
```

В приложении создан маршрут «/», который обрабатывает только GET-запросы и возвращает приветственное сообщение Hello World.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ nano api.py
```

Рисунок 11. Создание файла api.py

```
GNU nano 7.2 api.py
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def welcome() -> dict:
    return {"message": "Hello World"}
```

[Прочитано 6 строк]

^G Справка	^O Записать	^W Поиск	^K Вырезать	^T Выполнить	^C Позиция
^X Выход	^R ЧитФайл	^_ Замена	^U Вставить	^J Выводить	^/ К строке

Рисунок 12. Код приложения в файле api.py

Запуск приложения выполнен командой: `uvicorn api:app --port 8000 --reload`. Сервер запущен и ожидает запросы на адресе `http://127.0.0.1:8000`.

```
(venv) anna@nb-anna:~/Документы/Network_techn/todos$ uvicorn api:app --port 8000 --reload
INFO: Will watch for changes in these directories: ['/home/anna/Документы/Network_tech
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10911] using StatReload
INFO: Started server process [10913]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Рисунок 13. Запуск приложения uvicorn

Для проверки отправлен GET-запрос командой: `curl http://127.0.0.1:8000/`. Приложение вернуло ответ: `{"message":"Hello World"}`. Значит, приложение запущено и работает корректно.

```
anna@nb-anna: ~/Документы/Network_techn/todos x  anna@nb-anna: ~/Документы/Network_techn/todos x
anna@nb-anna:~/Документы/Network_techn/todos$ curl http://127.0.0.1:8000/
{"message":"Hello World"}anna@nb-anna:~/Документы/Network_techn/todos$
```

Рисунок 14. Тестирование приложения командой curl

В ходе работы создана виртуальная среда, освоено управление пакетами с помощью pip, настроен Docker и создано простое приложение FastAPI, которое успешно запущено и протестировано.